Язык программирования C

Лекция 6: стандартная библиотка, const

Лисица Никита

2026

Массивы и функции

```
int* get_array() {
  int array[100];
  for(int i = 0; i < 100; ++i) {
    array[i] = i;
  }
  return array;
}

int main() {
  int *array = get_array();
  printf("%d", array[0]);
}
```

- Что в этом коде не так?

Массивы и функции

```
int* get_array() {
  int array[100];
  for(int i = 0; i < 100; ++i) {
    array[i] = i;
  }
  return array;
}

int main() {
  int *array = get_array();
  printf("%d", array[0]);
}
```

- Что в этом коде не так? `array` – локальная переменная: она живёт на стеке, и будет “удалена” после выхода из функции `get_array()`

Массивы и функции

```
int* get_array() {
  int *array = malloc(100 * sizeof(int));
  for(int i = 0; i < 100; ++i) {
    array[i] = i;
  }
  return array;
}

int main() {
  int *array = get_array();
  printf("%d", array[0]);
  free(array); // Не забываем освободить память!
}
```

Массивы и функции

```
int* get_array() {
  int *array = malloc(100 * sizeof(int));
  for(int i = 0; i < 100; ++i) {
    array[i] = i;
  }
  return array;
}

int main() {
  int *array = get_array();
  printf("%d", array[0]);
  free(array); // Не забываем освободить память!
}
```

- `malloc` выделяет память в *куче*, мы сами контролируем момент её освобождения

Массивы и функции

```
int remove_odd(int *src, int size, int *dst) {
  int even_count = 0;
  for (int i = 0; i < size; ++i) {
    if ((src[i] % 2) == 0) ++even_count;
  }
  dst = malloc(even_count * sizeof(int));

  int *out = dst;
  for (int i = 0; i < size; ++i) {
    if ((src[i] % 2) == 0) *out++ = src[i];
  }

  return even_count;
}
```

- Что в этом коде не так?

Массивы и функции

```
int remove_odd(int *src, int size, int *dst) {
  int even_count = 0;
  for (int i = 0; i < size; ++i) {
    if ((src[i] % 2) == 0) ++even_count;
  }
  dst = malloc(even_count * sizeof(int));

  int *out = dst;
  for (int i = 0; i < size; ++i) {
    if ((src[i] % 2) == 0) *out++ = src[i];
  }

  return even_count;
}
```

- Что в этом коде не так? `dst` – аргумент функции, в которую `result` передаётся по значению: мы перезаписали `dst`, но в `result` осталось старое значение (`NULL`)

Массивы и функции

```
int remove_odd(int *src, int size, int **dst) {
  int even_count = 0;
  for (int i = 0; i < size; ++i) {
    if ((src[i] % 2) == 0) ++even_count;
  }
  *dst = malloc(even_count * sizeof(int));

  int *out = *dst;
  for (int i = 0; i < size; ++i) {
    if ((src[i] % 2) == 0) *out++ = src[i];
  }

  return even_count;
}
```

Ключевое слово const: переменные

```
const int page_size = 4096;
const float pi = 3.1415926535f;
```

Ключевое слово const: переменные

```
const int page_size = 4096;
const float pi = 3.1415926535f;
```

- **const**-переменной нельзя присвоить другое значение

Ключевое слово const: переменные

```
const int page_size = 4096;
const float pi = 3.1415926535f;
```

- **const**-переменной нельзя присвоить другое значение
- Защищает программиста от ошибок

Ключевое слово const: переменные

```
const int page_size = 4096;
const float pi = 3.1415926535f;
```

- **`const`**-переменной нельзя присвоить другое значение
- Защищает программиста от ошибок
- Лучше выражает намерение автора кода

Ключевое слово const: переменные

```
const int page_size = 4096;
const float pi = 3.1415926535f;
```

- **const**-переменной нельзя присвоить другое значение
- Защищает программиста от ошибок
- Лучше выражает намерение автора кода
- Облегчает оптимизацию программы для компилятора

Ключевое слово const: аргументы функций

```
void print(const int value) {
  ...
}
```

Ключевое слово const: аргументы функций

```
void print(const int value) {
  ...
}
```

- **const** относится к самому аргументу функции, т.е. к локальной переменной (а не к тому значению, которое было в него скопировано)

Ключевое слово const: аргументы функций

```
void print(const int value) {
  ...
}
```

- **const** относится к самому аргументу функции, т.е. к локальной переменной (а не к тому значению, которое было в него скопировано)
- Как правило, смысла в этом мало – вызывающего функцию не очень волнует, что произойдёт с *копией* его данных

Ключевое слово const: указатели

- В случае указателей **const** может быть как сам указатель, так и данные, на которые он указывает

```
int i = 10, j = 15;

const int * pi = &i; // Указатель на константу
*pi = 20;            // Ошибка: нельзя изменить константное значение
pi = &j;             // Можно переприсвоить

int * const pi = &i; // Константный сам указатель
*pi = 20;            // Можно изменить значение по указателю
pi = &j;             // Нельзя переприсвоить

// Нельзя переприсвоить, нельзя перезаписать значение по указателю
const int * const pi = &i;
```

Строковые литералы

- Следующие два способа инициализации строки делают радикально разные вещи:

```
char * str1 = "Hello!";
char str2[] = "World!";
```

Строковые литералы

- Следующие два способа инициализации строки делают радикально разные вещи:

```
char * str1 = "Hello!";
char str2[] = "World!";
```

- Первый создаёт указатель на строку

Строковые литералы

- Следующие два способа инициализации строки делают радикально разные вещи:

```
char * str1 = "Hello!";
char str2[] = "World!";
```

- Первый создаёт указатель на строку
- Вторый создаёт массив символов с этой строкой

Строковые литералы

- Строки, полученные первым способом, хранятся вместе с глобальными переменными программы, и их нельзя менять:

```
char * str = "Hello!";
str[0] = 'A'; // Ошибка выполнения
```

Строковые литералы

- Строки, полученные первым способом, хранятся вместе с глобальными переменными программы, и их нельзя менять:

```
char * str = "Hello!";
str[0] = 'A'; // Ошибка выполнения
```

- Строки, полученные втором способом, хранятся на стеке (как обычные переменные), и их можно менять:

```
char str[] = "World!";
str[0] = 'A'; // Ошибки нет
```

Строковые литералы

- Чтобы на это не попадаться, лучше объявлять такие указатели как **const**:

```
const char * str = "Hello!";

str[0] = 'A'; // Ошибка компиляции: указатель на const
```

Ключевое слово const: указатели в аргументах функций

```
size_t strlen(const char * str) {
  ...
}
```

- Есть ли в этом смысл?

Ключевое слово const: указатели в аргументах функций

```
size_t strlen(const char * str) {
  ...
}
```

- Есть ли в этом смысл? Да: функция явно говорит, что не изменит данные по нашему указателю, что в неё можно передать строковой литерал, и т.п.

Ключевое слово const: указатели в аргументах функций

```
size_t strlen(const char * str) {
  ...
}
```

- Есть ли в этом смысл? Да: функция явно говорит, что не изменит данные по нашему указателю, что в неё можно передать строковой литерал, и т.п.
- В таких случаях лучше всегда указывать **const**!

East const vs const west

- Ключевое слово **`const`** можно писать как слева, так и справа от типа – для компилятора разницы нет:

```
// Одно и то же:
const int x = 10;
int const y = 20;

// Одно и то же:
const int * p = &x;
int const * q = &y;
```

East const vs const west

- Ключевое слово **`const`** можно писать как слева, так и справа от типа – для компилятора разницы нет:

```
// Одно и то же:
const int x = 10;
int const y = 20;

// Одно и то же:
const int * p = &x;
int const * q = &y;
```

- **`const`** справа называют *east const* (потому восток тоже справа на большинстве карт)
- **`const`** слева называют *const west* (потому запад слева)
- Оба варианта распространены, это дело вкуса

Стандартная библиотека: stdlib.h

- `int atoi(const char *)` – конвертация строки в десятичное число:

```c
int x = atoi("123");
```

Стандартная библиотека: stdlib.h

- `int atoi(const char *)` – конвертация строки в десятичное число:

```
int x = atoi("123");
```

- Есть `atol` для `long` и `atoll` для `long long`

Стандартная библиотека: stdlib.h

- `int atoi(const char *)` – конвертация строки в десятичное число:

```
int x = atoi("123");
```

- Есть `atol` для `long` и `atoll` для `long long`
- Возвращает 0, если не смог сконвертировать (конкретную ошибку не выдаёт)

Стандартная библиотека: stdlib.h

- `int atoi(const char *)` – конвертация строки в десятичное число:

```
int x = atoi("123");
```

- Есть `atol` для `long` и `atoll` для `long long`
- Возвращает 0, если не смог сконвертировать (конкретную ошибку не выдаёт)
- Undefined behavior, если значение не влезает в тип возвращаемого значения

Стандартная библиотека: stdlib.h

- `long strtol(const char *str, char** str_end, int base)` – конвертация строки в число в любой системе счисления

Стандартная библиотека: stdlib.h

- `long strtol(const char *str, char** str_end, int base)` – конвертация строки в число в любой системе счисления
- Указатель на первый символ, не вошедший в число, записывается в `str_end`

Стандартная библиотека: stdlib.h

- `long strtol(const char *str, char** str_end, int base)` – конвертация строки в число в любой системе счисления
- Указатель на первый символ, не вошедший в число, записывается в `str_end`

```
char *str = "25hello";
char *end = NULL;
long x = strtol(str, &end, 10);
// end == str + 2
```

Стандартная библиотека: stdlib.h

- `int rand()` – генератор случайных чисел

Стандартная библиотека: stdlib.h

- `int rand()` – генератор случайных чисел
- Разные вызовы вернут разные числа $\Longrightarrow$ использует внутри какое-то глобальное состояние

Стандартная библиотека: stdlib.h

- `int rand()` – генератор случайных чисел
- Разные вызовы вернут разные числа $\Longrightarrow$ использует внутри какое-то глобальное состояние
- `void srand(unsigned int)` – проинициализировать состояние генератора

Стандартная библиотека: stdlib.h

- `int rand()` – генератор случайных чисел
- Разные вызовы вернут разные числа $\Longrightarrow$ использует внутри какое-то глобальное состояние
- `void srand(unsigned int)` – проинициализировать состояние генератора
- При инициализации через `srand()` одним и тем же значением гарантируется, что `rand()` будет выдавать одну и ту же последовательность

Стандартная библиотека: stdlib.h

- `int rand()` – генератор случайных чисел
- Разные вызовы вернут разные числа $\Longrightarrow$ использует внутри какое-то глобальное состояние
- `void srand(unsigned int)` – проинициализировать состояние генератора
- При инициализации через `srand()` одним и тем же значением гарантируется, что `rand()` будет выдавать одну и ту же последовательность
- Не потокобезопасные функции! Лучше избегать глобального состояния и пользоваться специализированными библиотеками

Стандартная библиотека: stdlib.h

- Быстрая сортировка:

```
void qsort (void* ptr, size_t count, size_t size,
  int (*compare)(const void*, const void*));
```

Стандартная библиотека: stdlib.h

- Быстрая сортировка:

```
void qsort (void* ptr, size_t count, size_t size,
  int (*compare)(const void*, const void*));
```

- Двоичный поиск:

```
void* bsearch(const void *key, const void *ptr, size_t count,
  size_t size, int (*compare)(const void*, const void*));
```

Стандартная библиотека: stdlib.h

- `int system(const char *)` – выполнить консольную команду и вернуть её код возврата

Стандартная библиотека: stdlib.h

- `int system(const char *)` – выполнить консольную команду и вернуть её код возврата
- `void exit(int)` – завершить программу с кодом возврата

Стандартная библиотека: stdlib.h

- `int system(const char *)` – выполнить консольную команду и вернуть её код возврата
- `void exit(int)` – завершить программу с кодом возврата
- `void abort()` – завершить программу аварийно (не flush'ит открытые файлы и т.п.)

Стандартная библиотека: string.h

- `void* memcpy(void *dst, const void *src, size_t count)` – копирует `count` байт из `src` в `dst`

Стандартная библиотека: string.h

- `void* memcpy(void *dst, const void *src, size_t count)` – копирует `count` байт из `src` в `dst`
- Крайне оптимизирована, напр. копирует большими кусками через SSE регистры

Стандартная библиотека: string.h

- `void* memcpy(void *dst, const void *src, size_t count)` – копирует `count` байт из `src` в `dst`
- Крайне оптимизирована, напр. копирует большими кусками через SSE регистры
- `char* strcpy(char *dest, const char *src)` – скопировать строку (размер понимается через нуль-терминатор)

Стандартная библиотека: string.h

- `char* strtok(char *str, const char *delim)` – разбивает строку на части, разделённые символами из строки `delim`

Стандартная библиотека: string.h

- `char* strtok(char *str, const char *delim)` – разбивает строку на части, разделённые символами из строки `delim`
- Первый вызов задаёт строку для разбиения, последующие вызовы делаются с `str = NULL`

Стандартная библиотека: string.h

- `char* strtok(char *str, const char *delim)` – разбивает строку на части, разделённые символами из строки `delim`
- Первый вызов задаёт строку для разбиения, последующие вызовы делаются с `str = NULL`
- Каждый вызов выдаёт начало следущей части, в её конец `strtok` запишет нулевой символ (т.е. он меняет входную строку!)

Стандартная библиотека: string.h

- `char* strtok(char *str, const char *delim)` – разбивает строку на части, разделённые символами из строки `delim`
- Первый вызов задаёт строку для разбиения, последующие вызовы делаются с `str = NULL`
- Каждый вызов выдаёт начало следущей части, в её конец `strtok` запишет нулевой символ (т.е. он меняет входную строку!)
- Запоминает текущую строку и позицию в ней в статической переменной внутри

Стандартная библиотека: string.h

- `char* strtok(char *str, const char *delim)` – разбивает строку на части, разделённые символами из строки `delim`
- Первый вызов задаёт строку для разбиения, последующие вызовы делаются с `str = NULL`
- Каждый вызов выдаёт начало следущей части, в её конец `strtok` запишет нулевой символ (т.е. он меняет входную строку!)
- Запоминает текущую строку и позицию в ней в статической переменной внутри

```c
char str[] = "To be, or not to be";
char delim[] = " ,";
char *p = strtok(str, delim);
while (p != NULL) {
  printf("%s\n", p);
  p = strtok(NULL, delim);
}
```

Стандартная библиотека: time.h

- `time_t` – тип, хранящий время в секундах

Стандартная библиотека: time.h

- `time_t` – тип, хранящий время в секундах
- Почти всегда – целочисленный: число секунд с 1 января 1970 (POSIX time)

Стандартная библиотека: time.h

- `time_t` – тип, хранящий время в секундах
- Почти всегда – целочисленный: число секунд с 1 января 1970 (POSIX time)
- `time_t time(time_t *)` – возвращает текущее время в секундах

Стандартная библиотека: time.h

- `time_t` – тип, хранящий время в секундах
- Почти всегда – целочисленный: число секунд с 1 января 1970 (POSIX time)
- `time_t time(time_t *)` – возвращает текущее время в секундах
- Принимает указатель по историческим причинам, можно туда передавать `NULL`

Стандартная библиотека: time.h

- `time_t` – тип, хранящий время в секундах
- Почти всегда – целочисленный: число секунд с 1 января 1970 (POSIX time)
- `time_t time(time_t *)` – возвращает текущее время в секундах
- Принимает указатель по историческим причинам, можно туда передавать `NULL`

```
time_t current_time = time(NULL);
printf("POSIX time: %llu", unsigned long long(current_time));
```

Стандартная библиотека: time.h

- `clock_t` – тип, хранящий *clock time* – время, затраченное процессором на нашу программу

Стандартная библиотека: time.h

- `clock_t` – тип, хранящий *clock time* – время, затраченное процессором на нашу программу
- Точность не специфицируется; можно сконвертировать в секунды через константу `CLOCKS_PER_SEC`

Стандартная библиотека: time.h

- `clock_t` – тип, хранящий *clock time* – время, затраченное процессором на нашу программу
- Точность не специфицируется; можно сконвертировать в секунды через константу `CLOCKS_PER_SEC`
- `clock_t clock()` – возвращает текущее время в `clock_t`

Стандартная библиотека: time.h

- `clock_t` – тип, хранящий *clock time* – время, затраченное процессором на нашу программу
- Точность не специфицируется; можно сконвертировать в секунды через константу `CLOCKS_PER_SEC`
- `clock_t clock()` – возвращает текущее время в `clock_t`
- Если в программе несколько потоков, вернёт суммарное время, затраченное на всех!

Стандартная библиотека: time.h

- **`clock_t`** – тип, хранящий *clock time* – время, затраченное процессором на нашу программу
- Точность не специфицируется; можно сконвертировать в секунды через константу `CLOCKS_PER_SEC`
- **`clock_t`** `clock()` – возвращает текущее время в **`clock_t`**
- Если в программе несколько потоков, вернёт суммарное время, затраченное на всех!

```
clock_t start = clock();
big_slow_function();
clock_t end = clock();
long long duration_in_seconds = (end - start) / CLOCKS_PER_SEC;
```

Стандартная библиотека: time.h

- `struct timespec` – структура, хранящая время/длительность как пару из секунд и наносекунд

Стандартная библиотека: time.h

- `struct timespec` – структура, хранящая время/длительность как пару из секунд и наносекунд
- `int timespec_get(struct timespec *result, int base)` – вернуть текущее время в часовом поясе / временной системе `base`

Стандартная библиотека: time.h

- `struct timespec` – структура, хранящая время/длительность как пару из секунд и наносекунд
- `int timespec_get(struct timespec *result, int base)` – вернуть текущее время в часовом поясе / временной системе `base`

```c
struct timespec current_time;
timespec_get(&current_time, TIME_UTC);

long long seconds     = current_time.tv_sec ;
long long nanoseconds = current_time.tv_nsec;
```

Стандартная библиотека: assert.h

- `assert(...)` – макрос, проверяющий переданное условие

Стандартная библиотека: assert.h

- `assert(...)` – макрос, проверяющий переданное условие
- В случае невыполнения условия в дебажной сборке печатает сообщение об ошибке и завершает программу

Стандартная библиотека: assert.h

- `assert(...)` – макрос, проверяющий переданное условие
- В случае невыполнения условия в дебажной сборке печатает сообщение об ошибке и завершает программу
- В релизной сборке ничего не делает

Стандартная библиотека: assert.h

- `assert(...)` – макрос, проверяющий переданное условие
- В случае невыполнения условия в дебажной сборке печатает сообщение об ошибке и завершает программу
- В релизной сборке ничего не делает
- Позволяет на этапе разработки отловить ошибки, которые дорого проверять в релизной сборке

Стандартная библиотека: assert.h

- `assert(...)` – макрос, проверяющий переданное условие
- В случае невыполнения условия в дебажной сборке печатает сообщение об ошибке и завершает программу
- В релизной сборке ничего не делает
- Позволяет на этапе разработки отловить ошибки, которые дорого проверять в релизной сборке

```
void print_array(int *array, int size) {
  assert(array != NULL);
  ...
}
```

Стандартная библиотека: assert.h

- Как `assert()` понимает, какая сейчас сборка?

Стандартная библиотека: assert.h

- Как `assert()` понимает, какая сейчас сборка?
- В релизной сборке компилятор добавляет глобальный макрос `NDEBUG` (*Not DEBUG*)

Стандартная библиотека: assert.h

- Как `assert()` понимает, какая сейчас сборка?
- В релизной сборке компилятор добавляет глобальный макрос `NDEBUG` (*Not DEBUG*)
- В дебажной сборке не добавляет :)

Стандартная библиотека: assert.h

- `assert()` стоит использовать только для ошибок программиста / логики программы, а не для валидации данных или ошибок окружения

Стандартная библиотека: assert.h

- `assert()` стоит использовать только для ошибок программиста / логики программы, а не для валидации данных или ошибок окружения

```c
FILE *f = fopen(...);

// Плохая идея! Ошибка от нас не зависит.
// В релизной сборке такую проверку тоже
// нужно делать!
assert(f != NULL);
```

Стандартная библиотека: stdint.h

`int8_t`	`uint8_t`
`int16_t`	`uint16_t`
`int32_t`	`uint32_t`
`int64_t`	`uint64_t`
`intmax_t`	`uintmax_t`
`intptr_t`	`uintptr_t`

Стандартная библиотека: stdint.h

`int8_t`	`uint8_t`
`int16_t`	`uint16_t`
`int32_t`	`uint32_t`
`int64_t`	`uint64_t`
`intmax_t`	`uintmax_t`
`intptr_t`	`uintptr_t`

- `int32_t` и т.п. – гарантированно (без-)знаковый, ровно 32 бита (не скомпилируется, если платформа такой тип не поддерживает)

Стандартная библиотека: stdint.h

`int8_t`	`uint8_t`
`int16_t`	`uint16_t`
`int32_t`	`uint32_t`
`int64_t`	`uint64_t`
`intmax_t`	`uintmax_t`
`intptr_t`	`uintptr_t`

- `int32_t` и т.п. – гарантированно (без-)знаковый, ровно 32 бита (не скомпилируется, если платформа такой тип не поддерживает)
- `intmax_t` – максимальный поддерживаемый тип

Стандартная библиотека: stdint.h

`int8_t`	`uint8_t`
`int16_t`	`uint16_t`
`int32_t`	`uint32_t`
`int64_t`	`uint64_t`
`intmax_t`	`uintmax_t`
`intptr_t`	`uintptr_t`

- `int32_t` и т.п. – гарантированно (без-)знаковый, ровно 32 бита (не скомпилируется, если платформа такой тип не поддерживает)
- `intmax_t` – максимальный поддерживаемый тип
- `intptr_t` – тип, гарантированно вмещающий любой указатель

Стандартная библиотека: stdint.h

`int8_t`	`uint8_t`
`int16_t`	`uint16_t`
`int32_t`	`uint32_t`
`int64_t`	`uint64_t`
`intmax_t`	`uintmax_t`
`intptr_t`	`uintptr_t`

- `int32_t` и т.п. – гарантированно (без-)знаковый, ровно 32 бита (не скомпилируется, если платформа такой тип не поддерживает)
- `intmax_t` – максимальный поддерживаемый тип
- `intptr_t` – тип, гарантированно вмещающий любой указатель
- Полезно использовать, если важен размер переменной (для сериализации, передачи на GPU, и т.д.)